

Toward a resolution of US7 and US14: Integrating template deduction for class templates into the standard library

Introduction

National body comments US7 and US14 request analysis of the standard library to determine what changes might be desirable in light of the C++17 adoption of [P0091R3](#) (Template argument deduction for class templates (rev. 6)). In this paper, we perform such an analysis and recommend wording changes for the standard library clauses.

Background

General background is provided in [P0433R0](#). We list here new items not reflected in that paper.

- A sample implementation is available at <https://github.com/mspertus/p0433>.
- P0091R4 discusses some possible language extensions. Our recommendations in this paper for integrating template deduction for class templates are not affected in any way by the adoption or non-adoption of the extensions in that paper. However, it should be noted that these language extensions were motivated by experience with the standard library. See the next bullet as well as the discussion of `valarray` below for more information.
- As has been noted multiple times on the reflector, scenarios like the following can arise with standard library types that act as containers for values (e.g., `optional`):

```
optional o(3); // Deduces optional<int>
optional o2 = o; // Deduces optional<int>
```

However, note that writers of template libraries may have to continue to specify some object constructions in the traditional fashion as illustrated here:

```
template<typename T> void foo(T t)
{
    optional o = t; // T=optional<int> => o is optional<int>, which may not be desired in generic code
    optional<T> o2 = t; // o2 is optional<optional<int>>
}
```

We feel this is the right tradeoff as application programmers consuming template libraries get full benefit of deduction and template library writers are not worse off than in C++14. P0091R4 suggests a language extension to extend deduction to the example above. However, note that this does not impact the present proposal.

Wording

The remainder of this paper walks through the standard library clauses and proposes wording changes to take advantage of template deduction for class templates. Even where no wording changes are required, we sometimes call out classes that benefit from implicitly generated deduction guides. Note that library vendors may choose to add additional explicit deduction guides to maintain the “as-if” rule. See §21 below for an example of this.

§17 [library]

This clause requires no changes because it specifies only C libraries and so no class templates are involved.

§18 [language.support]

No changes are required to §18 [language.support].

Rationale: All of the headers in §18 are listed in “Table 30 — Language support library summary” in §18.1 [support.general]:

--	--	--

	Subclause	Header(s)
18.2	Common definitions	<cstdint>
18.3	Implementation properties	<limits> <climits> <cfloat>
18.4	Integer types	<cstdint>
18.5	Start and termination	<cstdlib>
18.6	Dynamic memory management	<new>
18.7	Type identification	<typeinfo>
18.8	Exception handling	<exception>
18.9	Initializer lists	<initializer_list>
18.10	Other runtime support	<csignal> <csetjmp> <stdalign> <stdarg> <stdbool> <cstdlib>

<limits>

The only class template in <limits> is `numeric_limits`. It has no non-default constructors (and only static members), so no changes are needed.

<initializer_list>

The <initializer_list> class template has only a default constructor and “magic” construction (from braced initializer lists) that already does template constructor deduction, so no changes are required.

Remaining headers in this clause

None of the <c...> headers or <new>, <typeinfo>, or <exception> define class templates, so they are unaffected by template constructor deduction.

§19 [diagnostics]

No changes are required to §19 [diagnostics].

Rationale: All of the headers in §19 are listed in “Table 31 — Diagnostics library summary” in §19.1:

	Subclause	Header(s)
19.2	Exception classes	<stdexcept>
19.3	Assertions	<cassert>
19.4	Error numbers	<cerrno>
19.5	System error support	<system_error>

<stdexcept>, <cassert>, <cerrno>

None of these define any class templates, so they are unaffected by template constructor deduction.

<system_error>

This header defines the class templates `is_error_code_enum`, `is_error_condition_enum`, and specializations of `std::hash`, all of which have only default constructors.

§20 [utilities]

No changes are required to §19 [diagnostics].

Rationale: All of the headers in §20 are listed in “Table 32 — General utilities library summary” in §20.1:

	Subclause	Header(s)
20.2		<utility>

	Utilities components	
20.3	Compile-time integer sequences	<utility>
20.4	Pairs	<utility>
20.5	Tuples	<tuple>
20.6	Optional objects	<optional>
20.7	Variants	<variant>
20.8	Storage for any type	<any>
20.9	Fixed-size sequences of bits	<bitset>
20.10	Memory	<memory> <cstdlib>
20.11	Smart pointers	<memory>
20.12	Memory resources	<memory_resource>
20.13	Scoped allocators	<scoped_allocator>
20.14	Function objects	<functional>
20.15	Type traits	<type_traits>
20.16	Compile-time rational arithmetic	<ratio>
20.17	Time utilities	<chrono> <ctime>
20.18	Type indexes	<typeindex>
20.19	Execution policies	<execution>

§20.2 [utility]

No changes are required in this subclause as it specifies no class templates.

§20.3 [intseq]

No changes are required in §20.3. Although `integer_sequence` is a class template, it has no non-default constructors. Interestingly, while the name `make_integer_sequence` “looks like” a make function, which is what the paper is trying to replace, it is actually a type and therefore a different beast altogether.

§20.4 [pairs]

Per LWG, we no longer propose that the `pair` constructor unwrap `reference_wrapper`. Make the following change to the definition of `std::pair` in §20.4.2 [pairs.pair]:

```
void swap(pair& p) noexcept(see below);
};

template<class T1, class T2>
pair(T1&&, T2&&) -> pair<&decay_t<T1>, &decay_t<T2>>;
}
```

§20.5 [tuple]

`tuple` is handled similarly to `pair`. To accomplish that, make the following change to the definition of `std::tuple` in §20.5.2 [tuple.tuple]:

```
void swap(tuple&) noexcept(see below);
};

template<class... UTypes>
tuple(UTypes...) -> tuple<decay_t<UTypes>...>;
}
```

§20.6 [optional]

`optional` is handled similarly to `pair`. Modify §20.6.3 [optional.optional] as follows:

```
T *val; // exposition only
};

template<class T> optional(T&& t) -> optional<decay_t<T>>;
```

Note: See the [Background](#) above for additional discussion of optional.

\$20.7 [variant]

No changes required in this subclause. Note that we do not expect variants to leverage template type deduction for constructors (outside of copy and move constructors) because their *raison d'être* is their ability to hold types beyond those they were initialized with.

\$20.8 [any]

No changes are required in \$20.8 as it defines no class templates.

\$20.9 [template.bitset]

No changes required in this subclause. It would be tempting to have `bitset("01101"s)` deduce `bitset<5>`. However, we feel this is best left for a future proposal, likely after a clearer consensus around compile-time strings has been achieved.

\$20.10 [memory]

`allocator` requires no changes as only the copy constructor is appropriate for deduction.

\$20.11 [smartptr]

We add a deduction guide to enable code such as

```
int *ip = new int();
unique_ptr uip{ip}; // Deduce unique_ptr<int>
```

Note that the template parameter still needs to be given explicitly for arrays.

```
int *ip = new int[5];
unique_ptr uip{ip}; // Error. Deduces unique_ptr<int>
unique_ptr aip<int[]>{ip}; // Ok
```

At the end of the `unique_ptr` class definition in \$20.11.1.2 [unique.ptr.single], add the following:

```
// disable copy from lvalue
unique_ptr(const unique_ptr&) = delete;
unique_ptr& operator=(const unique_ptr&) = delete;
};

// 20.11.1.3.5, deduction guides
template<class T> unique_ptr(T*) -> unique_ptr<T, default_delete<T>>;
template<class T, class V> unique_ptr(T*, V) -> unique_ptr<T, V>;
template<class U, class V> unique_ptr(U, V) -> unique_ptr<typename pointer_traits<V::pointer>::element_type, V>;
}
```

Add a section

20.11.1.3.5 `unique_ptr` deduction guides [unique.ptr.deduction.guides]

```
template<class T, class V>
unique_ptr(T*, V) -> unique_ptr<T, V>;
```

Remarks: This deduction guide shall not participate in overload resolution unless `V::pointer` is not valid or does not denote a type.

```
template<class U, class V>
unique_ptr(U, V) -> unique_ptr<typename pointer_traits<V::pointer>::element_type, V>;
```

Remarks: This deduction guide shall not participate in overload resolution unless `V::pointer` is valid and denotes a type.

For `shared_ptr`, make the following modifications to \$20.11.2.2 [util.smartptr.shared]:

```
template<class U> bool owner_before(const shared_ptr<U>& b) const;
template<class U> bool owner_before(const weak_ptr<U>& b) const;
};

template <class T> shared_ptr(T*) -> shared_ptr<T>;
template <class T, class D> shared_ptr(T*, D) -> shared_ptr<T>;
template <class T, class D, class A> shared_ptr(T*, D, A) -> shared_ptr<T>;
template <class T, class P> shared_ptr(const shared_ptr<T>, P*) -> shared_ptr<T>;
explicit template <class T> shared_ptr(const weak_ptr<T> &) -> shared_ptr<T>;
```

```
template <class T, class D> shared_ptr(unique_ptr<T, D>&&) -> shared_ptr<T>;
```

For `weak_ptr`, make the following modification to §20.11.2.3 [util.smartptr.weak]

```
template<class U> bool owner_before(const weak_ptr<U>& b) const;
};
```

```
template<class T> weak_ptr(const shared_ptr<T>&) -> weak_ptr<T>;
```

§20.12 [memory.resource]

No changes are required in this subclause as the `polymorphic_allocator<Tp>` constructors do not contain any information about `Tp` (except when constructed from other `polymorphic_allocator` objects, in which case deduction is properly done without any changes required)

§20.13 [allocator.adaptor]

We supply an explicit deduction guide so that `scoped_allocator_adaptor` can deduce its outer allocator. At the end of the definition of class `scoped_allocator_adaptor` in §20.13.1 [allocator.adaptor.syn], add

```
scoped_allocator_adaptor select_on_container_copy_construction() const;
};
```

```
template<class OuterAlloc, class... InnerAllocs> scoped_allocator_adaptor(OuterAlloc&&, const InnerAllocs&&)
-> scoped_allocator_adaptor<decay_t<OuterAlloc>, decay_t<InnerAllocs>...>;
```

If a different outer allocator is desired, then it can still be specified explicitly.

§20.14 [function.objects]

`reference_wrapper` benefits from template deduction for constructors with no wording changes. We suggest allowing function to deduce its template argument when initialized by a function or member pointer. Add the following deduction guides at the end of the class definition for `function` in §20.14.12.2 [func.wrap.func]

```
template<class T> const T* target() const noexcept;
```

```
};
```

```
template<class R, class... ArgTypes>
function(R (*)(ArgTypes...)) -> function<R(ArgTypes...)>;
template<class R, class C, class... ArgTypes>
function(R(C::*)(ArgTypes...)) -> function<R(C &, ArgTypes...)>;
template<class R, class C, class... ArgTypes>
function(R(C::*)(ArgTypes...) const) -> function<R(C const &, ArgTypes...)>;
template<class R, class C, class... ArgTypes>
function(R(C::*)(ArgTypes...) volatile) -> function<R(C volatile &, ArgTypes...)>;
template<class R, class C, class... ArgTypes>
function(R(C::*)(ArgTypes...) const volatile) -> function<R(C const volatile &, ArgTypes...)>;
template<class R, class C> function(R C::*) ->
function<R&(C &)>;
template<class R, class C>
function(const R C::*) -> function<const R&(C const &)>;
template<class R, class C>
function(volatile R C::*) -> function<volatile R&(volatile C &)>;
template<class R, class C>
function(const volatile R C::*) -> function<const volatile R&(const volatile C &)>;
```

Note: There are some arbitrary choices in the deductions for member pointers. We would be interested to see if the committee agrees with having deduction guides for member pointers in addition to function pointers, and if so, whether these are the right choices.

Since template constructor deduction can be used to construct objects of type `default_searcher`, `boyer_moore_searcher`, and `boyer_moore_horspool_searcher`, we propose getting rid of the searcher creation functions by modifying §20.14p2 as follows:

```
// 20.14.13 searchers:
template<class ForwardIterator, class BinaryPredicate = equal_to<>>
class default_searcher;
template<class RandomAccessIterator,
class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
class BinaryPredicate = equal_to<>>
class boyer_moore_searcher;
template<class RandomAccessIterator,
class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
class BinaryPredicate = equal_to<>>
```

```
class boyer_moore_horspool_searcher;
```

```
template<class ForwardIterator, class BinaryPredicate = equal_to<>>
default_searcher<ForwardIterator, BinaryPredicate>
make_default_searcher(ForwardIterator pat_first, ForwardIterator pat_last,
                      BinaryPredicate pred = BinaryPredicate());
template<class RandomAccessIterator,
         class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
         class BinaryPredicate = equal_to<>>
boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_searcher(
    RandomAccessIterator pat_first, RandomAccessIterator pat_last,
    Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());
template<class RandomAccessIterator,
         class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
         class BinaryPredicate = equal_to<>>
boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_horspool_searcher(
    RandomAccessIterator pat_first, RandomAccessIterator pat_last,
    Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());
```

// 20.14.14, hash function primary template:

Delete §20.14.13.1.1 [func.searchers.default.creation]

2014.13.2.1 boyer_moore_searcher creation functions [func.searcher.boyer_moore.creation]

```
template<class ForwardIterator, class BinaryPredicate = equal_to<>>
default_searcher<ForwardIterator, BinaryPredicate>
make_default_searcher(ForwardIterator pat_first, RandomAccessIterator pat_last,
                      BinaryPredicate pred = BinaryPredicate());
```

Effects: Equivalent to:

```
return default_searcher<ForwardIterator, BinaryPredicate>(pat_first, pat_last, pred);
```

Delete §20.14.13.2.1 [func.searchers.boyer_moore.creation]

2014.13.2.1 boyer_moore_searcher creation functions [func.searcher.boyer_moore.creation]

```
template<class RandomAccessIterator,
         class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
         class BinaryPredicate = equal_to<>>
boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_searcher(
    RandomAccessIterator pat_first, RandomAccessIterator pat_last,
    Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());
```

Effects: Equivalent to:

```
return boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>(
    pat_first, pat_last, hf, pred);
```

Delete §20.14.13.3.1 [func.searchers.boyer_moore_horspool.creation]

2014.13.2.1 boyer_moore_searcher creation functions [func.searcher.boyer_moore.creation]

```
template<class RandomAccessIterator,
         class Hash = hash<typename iterator_traits<RandomAccessIterator>::value_type>,
         class BinaryPredicate = equal_to<>>
boyer_moore_horspool_searcher<RandomAccessIterator, Hash, BinaryPredicate>
make_boyer_moore_horspool_searcher(
    RandomAccessIterator pat_first, RandomAccessIterator pat_last,
    Hash hf = Hash(), BinaryPredicate pred = BinaryPredicate());
```

Effects: Equivalent to:

```
return boyer_moore_searcher<RandomAccessIterator, Hash, BinaryPredicate>(
    pat_first, pat_last, hf, pred);
```

§20.15 [meta]

Since the classes in this subclause are designed primarily for compile-time metaprogramming, they do not have constructors that require special treatment as a result of template deduction for constructors.

§20.16 [ratio]

Since the classes in this subclause are designed primarily for compile-time metaprogramming, they do not have constructors that require special treatment as a result of template deduction for constructors.

§20.17 [time]

No wording changes necessary for §20.17 to leverage template deduction for constructors. Constructs like `duration(5L)` behave as expected from the compiler-generated implicit deduction guides..

§20.18 [type.index]

There are no class templates defined in this subclause, so nothing to do.

§20.19 [execpol]

The only class template in this subclause is the type trait `is_execution_policy`, so no changes are required as described in §20.15 above.

§21 [strings]

The string library can take advantage of template argument deduction for constructors with no wording changes. For example, `basic_string_view("foo")` correctly deduces that `charT` is `char`.

It is worth noting that, while no wording changes are required, a library vendor may choose to make code changes in accordance with the “as-if” rule (§1.9 footnote 5) as illustrated by the following example. Suppose a vendor has previously leveraged the “as-if” rule to implement the constructor

```
basic_string<charT, traits, Allocator>::basic_string(const charT* str, const Allocator& a = Allocator());
```

using the equivalent but non-deducible type `value_type` in place of `charT` as follows:

```
basic_string<charT, traits, Allocator>::basic_string(const value_type* str, const Allocator& a = Allocator());
```

The vendor could maintain this convention while continuing to satisfy the “as-if” rule in C++17 by adding the following *deduction-guide*

```
template<class charT, class traits = char_traits<charT>, class Allocator = allocator<charT>>
    basic_string(const charT*, const Allocator& = Allocator()) -> basic_string<charT, Allocator>;
```

Similar considerations apply to `basic_string_view`.

§22 [localization]

Add the following to the end of the definition of class `wstring_convert` in §22.3.3.2.2 [conversions.string]

```
    size_t cvtcount;           // exposition only
};

template<class Codecvt>
    wstring_convert(Codecvt *)
        -> wstring_convert<Codecvt, typename Codecvt::intern_type>;

template<class Codecvt>
    wstring_convert(Codecvt *, typename Codecvt::state_type)
        -> wstring_convert<Codecvt, typename Codecvt::intern_type>;
```

In addition to the above explicit deduction guide, `wstring_convert` and `wbuffer_convert` (§22.3.3.2.3) benefit from additional implicit deduction guides.

§23 [containers]

All of the headers in §23 are listed in “Table 102 — Containers library summary” in §23.1:

	Subclause	Header(s)
23.2	Requirements	
23.3	Sequence containers	<array> <deque> <forward_list> <list>

		<vector>
23.4	Associative containers	<map> <set>
23.5	Unordered associative containers	<unordered_map> <unordered_set>
23.6	Container adaptors	<queue> <stack>

The only wording change in §23 is to add deduction guides for constructing containers from iterators as described in “Explicitly specified Deduction Guides” in [P0091R3](#), but we have to make that change many times as follows:

§23.2 [container.requirements]

Modify §23.2.3p14 [sequence.reqmts] as follows:

For every sequence container defined in this clause and in clause 21:

- If the constructor

```
template <class InputIterator>
X(InputIterator first, InputIterator last,
  const allocator_type& alloc = allocator_type())
```

is called with a type `InputIterator` that does not qualify as an input iterator, then the constructor shall not participate in overload resolution.

- If the *deduction-guide*

```
template <class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
X(InputIterator, InputIterator,
  const Allocator& = Allocator())
-> X<typename iterator_traits<InputIterator>::value_type, Allocator>;
```

is called with a type `InputIterator` that does not qualify as an input iterator, then the *deduction-guide* shall not participate in overload resolution.

- If the member functions of the forms:

Add a paragraph to the end of §23.2.6 [associative.reqmts]

A deduction guide for an associative container shall not participate in overload resolution if it has an `InputIterator` template parameter that is called with a type that does not qualify as an input iterator, or if it has an `Allocator` template parameter that is called with a type that does not qualify as an allocator (§17.5.3.5 [allocator.requirements]), or if it has a `Compare` template parameter that is called with a type that does qualify as an allocator.

Add a paragraph to the end of §23.2.7 [unord.req]

A deduction guide for an unordered associative container shall not participate in overload resolution if any of the following are true.

- It has an `InputIterator` template parameter that is called with a type that does not qualify as an input iterator
- it has an `Allocator` template parameter that is called with a type that does not qualify as an allocator (§17.5.3.5 [allocator.requirements])
- It has a `Hash` template parameter that is called with a type that does qualify as an allocator
- It has a `Pred` template parameter that is called with a type that does qualify as an allocator

§23.3 [sequences]

At the end of the definition of class `deque` in §23.3.8.1 [deque.overview], add the following *deduction-guide*:

```
void clear() noexcept;
};
```

```
template <class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
deque(InputIterator, InputIterator,
  const Allocator& = Allocator())
-> deque<typename iterator_traits<InputIterator>::value_type, Allocator>;
```

At the end of the definition of class `forward_list` in §23.3.9.1 [forwardlist.overview], add the following *deduction-guide*:

```
void reverse() noexcept;
};
```

```
template <class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
```



```
forward_list(InputIterator, InputIterator,
             const Allocator& = Allocator())
-> forward_list<typename iterator_traits<InputIterator>::value_type, Allocator>;
```

At the end of the definition of class `list` in §23.3.10.1 [list.overview], add the following *deduction-guides*:

```
void reverse() noexcept;
};

template <class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
list(InputIterator first, InputIterator last,
     const Allocator& = Allocator())
-> list<typename iterator_traits<InputIterator>::value_type, Allocator>;
```

At the end of the definition of class `vector` in §23.3.11.1 [vector.overview], add the following *deduction-guide*:

```
void clear() noexcept;
};

template <class InputIterator, class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
vector(InputIterator, InputIterator,
       const Allocator& = Allocator())
-> vector<typename iterator_traits<InputIterator>::value_type, Allocator>;
```

§23.4 [associative]

First, we note that associative containers cannot always deduce their template parameters from an initializer list as illustrated by the following code.

```
map m = {"foo", 2}, {"bar", 3}, {"baz", 4}; // Error: initializer_list not reified in type system
map m2 = {pair{"foo", 2}, pair{"bar", 3}, pair{"baz", 4}}; // OK
```

Hopefully, this may be addressed by a future language extension in the post-C++17 timeline. Note also that we apply `remove_const_t` to the keys in order to find the proper comparator (or hash in case of unordered containers).

Proceeding to the wording changes, at the end of the definition of class `map` in §23.4.4.1 [map.overview], add the following *deduction-guides*:

```
template <class K>
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
         class Compare = default_order_t<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
         class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
map(InputIterator, InputIterator,
    const Compare& = Compare(), const Allocator& = Allocator())
-> map<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
      typename iterator_traits<InputIterator>::value_type::second_type,
      Compare, Allocator>;

template <class InputIterator, class Allocator>
map(InputIterator, InputIterator, const Allocator&)
-> map<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
      typename iterator_traits<InputIterator>::value_type::second_type,
      default_order_t<typename iterator_traits<InputIterator>::value_type::first_type>, Allocator>;
```

At the end of the definition of class `multimap` in §23.4.5.1 [multimap.overview], add the following *deduction-guides*:

```
template <class K>
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
         class Compare = default_order_t<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
         class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
multimap(InputIterator, InputIterator,
         const Compare& = Compare(), const Allocator& = Allocator())
-> multimap<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
           typename iterator_traits<InputIterator>::value_type::second_type,
           Compare, Allocator>;

template <class InputIterator, class Allocator>
multimap(InputIterator, InputIterator, const Allocator&)
-> multimap<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
           typename iterator_traits<InputIterator>::value_type::second_type,
           default_order_t<typename iterator_traits<InputIterator>::value_type::first_type>, Allocator>;
```

At the end of the definition of class `set` in §23.4.6.1 [set.overview], add the following *deduction-guide*:

```
template <class K>
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
    class Compare = default_order_t<typename iterator_traits<InputIterator>::value_type>,
    class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
set(InputIterator, InputIterator,
    const Compare& = Compare(), const Allocator& = Allocator())
-> set<typename iterator_traits<InputIterator>::value_type, Compare, Allocator>;

template <class InputIterator, class Allocator>
set(InputIterator, InputIterator, const Allocator&)
-> set<typename iterator_traits<InputIterator>::value_type,
    default_order_t<typename iterator_traits<InputIterator>::value_type>, Allocator>;
```

At the end of the definition of class `multiset` in §23.4.7.1 [multiset.overview], add the following *deduction-guide*:

```
template <class K>
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class InputIterator,
    class Compare = default_order_t<typename iterator_traits<InputIterator>::value_type>,
    class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
multiset(InputIterator, InputIterator,
    const Compare& = Compare(), const Allocator& = Allocator())
-> multiset<typename iterator_traits<InputIterator>::value_type, Compare, Allocator>;

template <class InputIterator, class Allocator>
multiset(InputIterator, InputIterator, const Allocator&)
-> multiset<typename iterator_traits<InputIterator>::value_type,
    default_order_t<typename iterator_traits<InputIterator>::value_type>, Allocator>;
```

§23.5 [unord]

At the end of the definition of class `unordered_map` in §23.5.4.1 [unord.map.overview], add the following *deduction-guides*:

```
void reserve(size_type n);
};

template <class InputIterator,
    class Hash = hash<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
    class Pred = std::equal_to<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
    class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
unordered_map(InputIterator, InputIterator,
    size_type = see below,
    const Hash& = Hash(), const Pred& = Pred(), const Allocator& = Allocator())
-> unordered_map<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
    typename iterator_traits<InputIterator>::value_type::second_type,
    Hash, Pred, Allocator>;

template <class InputIterator, class Allocator>
unordered_map(InputIterator, InputIterator, size_type, const Allocator&)
-> unordered_map<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
    typename iterator_traits<InputIterator>::value_type::second_type,
    hash<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
    std::equal_to<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
    Allocator>;

template <class InputIterator, class Hash, class Allocator>
unordered_map(InputIterator, InputIterator, size_type, const Hash&, const Allocator&)
-> unordered_map<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
    typename iterator_traits<InputIterator>::value_type::second_type, Hash,
    std::equal_to<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
    Allocator>;
```

At the end of the definition of class `unordered_multimap` in §23.5.5.1 [unord.multimap.overview], add the following *deduction-guide*:

```
void reserve(size_type n);
};

template <class InputIterator,
    class Hash = hash<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
    class Pred = std::equal_to<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
```

```

        class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>
unordered_multimap(InputIterator, InputIterator,
    size_type = see_below,
    const Hash& = Hash(), const Pred& = Pred(), const Allocator& = Allocator())
-> unordered_multimap<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
    typename iterator_traits<InputIterator>::value_type::second_type,
    typename iterator_traits<InputIterator>::value_type::first_type>>,
    Hash, Pred, Allocator>;

template <class InputIterator, class Allocator>
unordered_multimap(InputIterator, InputIterator, size_type, const Allocator&)
-> unordered_multimap<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
    typename iterator_traits<InputIterator>::value_type::second_type,
    hash<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>,
    std::equal_to<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>',
    Allocator>;

template <class InputIterator, class Hash, class Allocator>
unordered_multimap(InputIterator, InputIterator, size_type, const Hash&, const Allocator&)
-> unordered_multimap<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
    typename iterator_traits<InputIterator>::value_type::second_type, Hash,
    std::equal_to<remove_const_t<typename iterator_traits<InputIterator>::value_type::first_type>>, Allocator>;

```

At the end of the definition of class `unordered_set` in §23.5.6.1 [unord.set.overview], add the following *deduction-guides*:

```

void reserve(size_type n);
};

template <class InputIterator,
    class Hash = hash<typename iterator_traits<InputIterator>::value_type>,
    class Pred = std::equal_to<typename iterator_traits<InputIterator>::value_type>,
    class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
unordered_set(InputIterator, InputIterator,
    size_type = see_below,
    const Hash& = Hash(), const Pred& = Pred(), const Allocator& = Allocator())
-> unordered_set<typename iterator_traits<InputIterator>::value_type::first_type,
    Hash, Pred, Allocator>;

template <class InputIterator, class Allocator>
unordered_set(InputIterator, InputIterator, size_type, const Allocator&)
-> unordered_set<typename iterator_traits<InputIterator>::value_type,
    hash<typename iterator_traits<InputIterator>::value_type>,
    std::equal_to<typename iterator_traits<InputIterator>::value_type>,
    Allocator>;

template <class InputIterator, class Hash, class Allocator>
unordered_set(InputIterator, InputIterator, size_type, const Hash&, const Allocator&)
-> unordered_set<typename iterator_traits<InputIterator>::value_type, Hash,
    std::equal_to<typename iterator_traits<InputIterator>::value_type>, Allocator>;

```

At the end of the definition of class `unordered_multiset` in §23.5.7.1 [unord.multiset.overview], add the following *deduction-guides*:

```

void reserve(size_type n);
};

template <class InputIterator,
    class Hash = hash<typename iterator_traits<InputIterator>::value_type>,
    class Pred = std::equal_to<typename iterator_traits<InputIterator>::value_type>,
    class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
unordered_multiset(InputIterator, InputIterator,
    size_type = see_below,
    const Hash& = Hash(), const Pred& = Pred(), const Allocator& = Allocator())
-> unordered_multiset<typename iterator_traits<InputIterator>::value_type::first_type,
    Hash, Pred, Allocator>;

template <class InputIterator, class Allocator>
unordered_multiset(InputIterator, InputIterator, size_type,
    const Allocator& = Allocator())
-> unordered_multiset<typename iterator_traits<InputIterator>::value_type,
    hash<typename iterator_traits<InputIterator>::value_type>,
    std::equal_to<typename iterator_traits<InputIterator>::value_type>,
    Allocator>;

template <class InputIterator, class Hash, class Allocator>
unordered_multiset(InputIterator, InputIterator, size_type, const Hash&, const Allocator&)
-> unordered_multiset<typename iterator_traits<InputIterator>::value_type, Hash,
    std::equal_to<typename iterator_traits<InputIterator>::value_type>, Allocator>;

```

§23.6 [container.adaptors]

At the end of §23.6.1 [container.adaptors.general], and the following paragraph

A deduction guide for an container adaptor shall not participate in overload resolution if any of the following are true.

- It has an `InputIterator` template parameter that is called with a type that does not qualify as an input iterator
- It has a `Container` template parameter that does not qualify as a container
- It has an `Allocator` template parameter that is called with a type that does not qualify as an allocator (§17.5.3.5 [allocator.requirements])

At the end of the definition of class `queue` in §23.6.4.1 [queue.defn] insert

```
void swap(queue& q) noexcept(is_nothrow_swappable_v<Container>)
{ using std::swap; swap(c, q.c); }
};

explicit template<class Container> queue(const Container&&)
-> queue<typename decay_t<Container>::value_type>
explicit template<class Alloc> queue(const Alloc&)
-> queue<typename Alloc::value_type>
template<class Container, class Alloc> queue(Container&& const Alloc&)
-> queue<typename decay_t<Container>::value_type>
```

At the end of the definition of class `priority_queue` in §23.6.5 [priority.queue], add the following *deduction-guides*:

```
void swap(priority_queue& q) noexcept(is_nothrow_swappable_v<Container> &&
                                     is_nothrow_swappable_v<Compare>)
{ using std::swap; swap(c, q.c); swap(comp, q.comp); }
};

template < class Container, class Compare = default_order_t<typename Container::value_type>>
priority_queue(const Compare&, Container&&
-> priority_queue<typename Container::value_type, Container, Compare>;

template<class InputIterator,
        class Compare = default_order_t<typename iterator_traits<InputIterator>::value_type>,
        class Container = vector<typename iterator_traits<InputIterator>::value_type>>
priority_queue(InputIterator, InputIterator, const Compare& = Compare(), Container&& = Container())
-> priority_queue<typename iterator_traits<InputIterator>::value_type, Container>;

template<class Compare, class Alloc>
priority_queue(const Compare&, const Alloc&) -> priority_queue<typename Alloc::value_type>

template<class Compare, class Container, class Alloc>
priority_queue(const Compare&, Container&&, const Alloc&) -> priority_queue<typename Container::value_type, Container>
```

At the end of the definition of class `stack` in § 23.6.6.1 [stack.defn], add

```
void swap(stack& q) noexcept(is_nothrow_swappable_v<Container>)
{ using std::swap; swap(c, q.c); }
};

template<class Container>
stack(Container&& = Container()) -> stack<typename Container::value_type, Container>
template<class Container, class Alloc>
stack(Container&&, const Alloc &) -> stack<typename Container::value_type, Container>
```

§24 [iterators]

No changes are required in clause 24 as the implicitly generated deduction guides provide the necessary deduction.

§25 [algorithms]

No changes are required in clause 25 due to lack of template classes in this clause.

§26 [numerics]

We consider the sections with instantiable template classes.

§26.5 [complex.numbers]

Class `complex` does not require explicit deduction guides as the implicitly generated deduction guides provide the necessary deduction.

§26.6 [rand]

This section does not require explicit deduction guides. The random number engines (§ 26.6.3 [rand.eng]) all have non-type template parameters and are therefore not suitable for template parameter deduction. All of the distributions (§26.6.8 [rand.dist]) that could in principle be deduced from constructor arguments are properly deduced by their implicitly generated deduction guides.

§26.7 [numarray]

We consider `valarray`. We first note that the implicit deduction guides imply the following:

```
int iarr[] = {1, 2, 3};
int *ip = iarr;
    valarray va(ip, 3); // Deduces valarray<int>
```

The point is that the `valarray<T>::valarray(const T *, size_t)` constructor is a better match than the `valarray<T>::valarray(const T &, size_t)` constructor. We think this is preferable because we believe that is the much more common default. Note that P0091R4 discusses a language extension (= delete for the `valarray(const T *, size_t)` deduction guide) that would facilitate suppressing deduction in this case. However, as above, we believe the appropriate deduction is produced by the implicitly-generated deduction guides.

We do add a deduction guide to enable the following deduction:

```
int iarr[] = {1, 2, 3};
valarray va(iarr, 3); // Needs explicit deduction guide to deduce valarray<int>
```

At the end of the definition of class `valarray` in §26.7.2.1 [template.valarray.overview], insert the following

```
void resize(size_t sz, T c = T());
};

template<typename T, size_t cnt> valarray(T&[cnt], size_t) -> valarray<remove_const_t<T>>;
```

§27 [input.output]

No changes are required in clause 27 as the implicitly generated deduction guides provide the necessary deduction.

§28 [re]

At the end of the definition of class `basic_regex` in §28.8 [re.regex], insert the following:

```
// 28.8.6, swap
void swap(basic_regex&);
};

// 28.8.8 deduction guides
template <class ForwardIterator>
    basic_regex(ForwardIterator, ForwardIterator) -> basic_regex<typename iterator_traits<ForwardIterator>::value_type>;
template<class ForwardIterator>
    basic_regex(ForwardIterator, ForwardIterator, typename regex_constants::syntax_option_type)
        -> basic_regex<typename iterator_traits<ForwardIterator>::value_type>;
```

Insert a new section:

28.8.8 `basic_regex` deduction guides [re.regex.deduction.guides]

```
template <class ForwardIterator>
    basic_regex(ForwardIterator, ForwardIterator) -> basic_regex<typename iterator_traits<ForwardIterator>::value_type>;
template<class ForwardIterator>
    basic_regex(ForwardIterator, ForwardIterator, typename regex_constants::syntax_option_type)
        -> basic_regex<typename iterator_traits<ForwardIterator>::value_type>;
```

Remarks: These deduction guides shall not participate in overload resolution unless `ForwardIterator` is a valid iterator.

§29 [atomics]

No changes are required in clause 29 as the implicitly generated deduction guides provide the necessary deduction.

§30 [thread]

The implicitly generated deduction guides do the job except for `promise`. At the end of the definition of class `<promise>` in §30.6.5 [futures.promise], insert the following:

```
// setting the result with deferred notification
void set_vale_at_thread_exit(see below);
void set_exception_at_thread_exit(exception_ptr p);
};

template <class Alloc> promise(allocator_arg_t, const Alloc&)
    -> promise<typename Alloc::value_type>;

template <class R>
    void swap(promise<R>& x, promise<R>& y) noexcept;
```